

The Entropy System Guide

A Self-Documenting Interactive Textbook

Scott Duhadway

Learn how to use EntropyExplainer and EntropyEditor by reading this textbook inside EntropyExplainer. Every feature of the system is demonstrated by the content itself.

Version 1.0.0 | 2026-04-20

entropy, guide, meta, self-referential

Table of Contents

- Welcome to Entropy
 - The .entropy.json Format
 - Children: Building Hierarchy
 - Connections: Cross-Linking
 - Writing Content in Markdown
 - EntropyExplainer: The Reader
 - Themes and Reading Settings
 - Progress Tracking
 - Accessibility Features
 - EntropyEditor: The Authoring Tool
 - The Rich Text Toolbar
 - Managing Children and Connections
 - The Relationship Graph
 - Command Palette and Shortcuts
- Technical Architecture
- Editor State Management
- Reader State Management
- Quick Start: Your First Textbook
- Extending the System

● *intro* ● *intermediate* ● *advanced* ● *expert*

Welcome to Entropy

Intro · 3 min · overview, start-here

You are reading this inside the tool it describes. Everything you see here is a feature demonstration.

Entropy is two tools that share one content format:

EntropyExplainer is the reader you are using right now. It turns structured content into a choose-your-own-adventure textbook.

EntropyEditor is the authoring tool. It produces the `.entropy.json` files that this reader consumes.

This guide teaches you both tools by example. The sidebar on your left shows the full table of contents. The cards below let you choose what to explore first.

How to Navigate

You have several ways to move through this book:

Child cards below each section lead deeper into a topic

Connection pills at the bottom link to related topics sideways

Back and **Home** buttons retrace your steps

Prev / Next buttons follow the linear chapter order

Sidebar lets you jump anywhere

Keyboard: Alt+Left goes back, / focuses search, Ctrl+/ toggles the sidebar

Pick a path and start exploring. The blue dots in the sidebar will track where you have been.

See Also

Tangent [Skip ahead to the quick-start](#)

Deeper [Technical architecture overview](#)

In This Section

- [The .entropy.json Format](#)
- [EntropyExplainer: The Reader](#)
- [EntropyEditor: The Authoring Tool](#)

The .entropy.json Format

Intro · 5 min · format, json, schema

A single JSON file defines an entire textbook. Here is what goes inside it.

Every Entropy textbook is one `.entropy.json` file. The format has four top-level keys:

meta — book-level metadata (title, author, version, timestamps, description, tags)

root — the ID of the entry node (where readers start)

nodes — an object of all content nodes, keyed by their ID

themes — optional theme configuration for the reader

What Is a Node?

A node is one section of content. Think of it as a page, a chapter, or a concept — whatever granularity fits your material. Each node has:

id — a URL-safe slug like `supply-chain-basics`

title — what appears in the sidebar and at the top of the page

summary — a brief description shown on parent cards (like the italicized text you see below this title)

content — the body text, written in Markdown

children — an ordered list of child node IDs (the cards below this section)

connections — cross-links to other nodes with a type and optional label

media — images, video, audio, embeds, or diagrams

tags — free-form labels for categorization

difficulty — one of: intro, intermediate, advanced, expert

estimatedTime — a human-readable time estimate like "5 min"

order — numeric sort order among siblings

The schema is formally defined in `shared/schema.json` and is validated by both tools.

See Also

Related [How to create this format](#)

Deeper [Technical deep-dive](#)

In This Section

- [Children: Building Hierarchy](#)
- [Connections: Cross-Linking](#)
- [Writing Content in Markdown](#)

Children: Building Hierarchy

Intro · 4 min · hierarchy, tree, navigation

Children define the tree structure. They are ordered, and they render as the clickable cards you see below each section.

The `children` array on each node is an ordered list of node IDs. This creates the tree structure that powers:

The **sidebar navigation** (the indented list on the left)

The **child cards** (the clickable boxes below this text)

The **Prev/Next** linear order (depth-first walk of the tree)

The **breadcrumb trail** (the path shown above the title)

Ordering Matters

Children appear in the order listed. The first child is the first card, the first sidebar entry, and the first stop on the linear path.

Orphan Nodes

A node that is not any other node's child is an "orphan." Orphans still appear in the sidebar (at the bottom), but they are not part of the linear navigation flow. They are reachable via connections or direct search.

Circular References

The system handles cycles gracefully. If Node A lists Node B as a child, and Node B lists Node A, the tree renderer uses a visited-set to avoid infinite loops. The first encounter wins.

See Also

Related [Connections vs. children](#)

Related [Managing children in the editor](#)

Connections: Cross-Linking

Intermediate · 5 min · connections, cross-links, navigation

Connections are non-hierarchical links between nodes. They have types that indicate the nature of the relationship.

While children define the tree, connections define the web. They are the "choose your own adventure" links that let readers explore laterally.

Each connection has:

to — the target node ID

type — one of five relationship types

label — optional display text (defaults to the target's title)

Connection Types

The five types each render with a distinct color:

related (blue) — general association between topics

prerequisite (amber) — "you should read this first"

deeper (purple) — a more advanced take on the same topic

example (green) — a concrete illustration or case study

tangent (pink) — interesting but not essential; a rabbit hole

Look at the connection pills below this section. Each one demonstrates a different type. Click any to follow the link.

One-Way Links

Connections are directional. If Node A connects to Node B, that does not mean Node B connects back to A. You can create reciprocal links in the editor if you want bidirectional navigation.

See Also

Related [Children vs. connections](#)

Prerequisite [Review the full node schema](#)

Deeper [How connections render in the graph](#)

Example [Practical walkthrough](#)

Tangent [Adding new connection types](#)

Writing Content in Markdown

Intro · 6 min · markdown, formatting, syntax

Node content is written in Markdown. Both tools parse it — the editor gives you a rich text toolbar, the reader renders it to HTML.

The `content` field of each node is a Markdown string. Both tools parse it independently:

EntropyExplainer uses a custom parser (`js/markdown.js`) that handles headings, bold, italic, code, links, lists, blockquotes, code blocks, horizontal rules, and strikethrough.

EntropyEditor converts Markdown to HTML for its contenteditable WYSIWYG editor, and converts it back on save.

Supported Syntax

Here is every formatting option, demonstrated live:

Headings

This is an H3 heading (above). H1 and H2 also work.

Inline Formatting

Bold text is wrapped in double asterisks. *Italic text* uses single asterisks. `Inline code` uses backticks. [Links](#) use bracket-paren syntax. ~~Strikethrough~~ uses tildes.

Lists

Unordered:

First item

Second item

Third item

Ordered:

Step one

Step two

Step three

Blockquotes

This is a blockquote. Use it for callouts, quotations, or emphasis.

Code Blocks

```
{
  "id": "example-node",
  "title": "A Minimal Node",
  "content": "Hello, world.",
  "children": [],
  "connections": []
}
```

Horizontal Rules

A line of three dashes creates a separator:

All of these round-trip cleanly through the editor. What you write in Markdown is what you see in the reader.

See Also

Related [The toolbar that generates this Markdown](#)

Prerequisite [Back to the format overview](#)

EntropyExplainer: The Reader

Intro · 4 min · reader, explainer, navigation

The reader turns `.entropy.json` files into interactive, navigable textbooks. You are using it right now.

EntropyExplainer is a zero-dependency web application. Open `explainer/index.html` in any browser, load a `.entropy.json` file, and start reading.

Loading Content

Two ways to load a file:

File picker — click the file input on the landing page

Drag and drop — drag a `.entropy.json` file anywhere on the page

Once loaded, the sidebar tree appears on the left and the content area fills with the root node.

Navigation Model

The reader offers four navigation modes simultaneously:

Hierarchical — click child cards to go deeper, breadcrumbs to go up

Associative — click connection pills to jump sideways

Linear — use Prev/Next to follow the tree order like a traditional book

Random access — use the sidebar or search to jump anywhere

Readers choose their own path. A curious reader might follow tangent connections. A methodical reader might use Prev/Next. Both are valid.

See Also

Related [The tool that creates content for this reader](#)

Prerequisite [The format this reader consumes](#)

In This Section

- [Themes and Reading Settings](#)
- [Progress Tracking](#)
- [Accessibility Features](#)

Themes and Reading Settings

Intro · 3 min · themes, customization, typography

Four visual themes and adjustable typography. Try them — your preferences persist across sessions.

Click the palette icon in the header to open the theme picker. Four themes are available:

Clean — light background, minimal decoration, optimized for readability

Academic — warm tones, serif-influenced, designed for long study sessions

Dark — dark background, reduced eye strain for night reading

High Contrast — maximum contrast for accessibility

Reading Settings

Click the font icon (Aa) in the header to adjust:

Font size — 14px to 24px

Line spacing — 1.4 to 2.2

Content width — 500px to 900px

All settings are saved to localStorage and restored automatically on your next visit.

Theme Persistence

Your chosen theme persists across different books. If you prefer Dark mode, every book you open will use it until you change it.

See Also

Related [More accessibility features](#)

Deeper [How to create custom themes](#)

Progress Tracking

Intro · 3 min · progress, tracking, persistence

The reader remembers where you have been and where you stopped. Per-book, persistent across sessions.

Entropy tracks your reading progress in two ways:

Visited Indicators

Every section you visit gets marked. Look at the sidebar — sections you have read show a blue dot instead of the default gray one. The child cards on parent pages also show a small blue indicator for visited sections.

This helps you see at a glance what you have covered and what remains.

Progress Arc

The circular icon in the header fills as you visit more sections. Hover over it to see your exact percentage and count.

Position Persistence

If you close the tab and reopen this book later, the reader will return you to the last section you were reading. This is tracked per-book (by title slug) in localStorage.

Privacy

All tracking is local to your browser. No data is sent anywhere. Clearing your browser's localStorage resets all progress.

See Also

[Related](#) [Other persistent settings](#)

Accessibility Features

Intermediate · 4 min · `accessibility`, `a11y`, `keyboard`, `screen-reader`

Keyboard navigation, screen reader support, reduced motion, high contrast, and print styles.

Entropy is designed to be usable by everyone.

Keyboard Navigation

Shortcut	Action
Alt + Left	Go back
Alt + Home	Go to root
/	Focus search
Ctrl + /	Toggle sidebar
Escape	Close modals
Tab	Navigate focusable elements

Screen Readers

All navigation elements have ARIA roles and labels

An `aria-live` region announces page transitions

Skip-to-content link is available

Interactive elements have descriptive `aria-label` attributes

Reduced Motion

If your OS is set to reduce motion (`prefers-reduced-motion`), all animations and transitions are disabled.

Forced Colors / High Contrast

The High Contrast theme works with Windows High Contrast Mode and `forced-colors` media queries.

Print Styles

Printing a page produces a clean, ink-friendly layout with hidden navigation elements.

See Also

Related [Theme and font settings](#)

EntropyEditor: The Authoring Tool

Intro · 4 min · editor, authoring, workspace

A three-panel editor for creating and modifying .entropy.json files. Dark mode, keyboard-driven, minimal clicks.

EntropyEditor is where you build your textbooks. Open `editor/index.html` in a browser and you get a three-panel workspace:

Left panel — the node tree, showing your book's structure

Center panel — the node editor, where you write content

Right panel — book metadata, live preview, and an interactive graph

Getting Started

Click **New** to create a blank textbook, or **Open** to load an existing `.entropy.json` file

Select a node in the left tree to begin editing

Write content in the rich text editor

Add children, connections, and media using the inline forms

Click **Save** to download your `.entropy.json` file

Design Philosophy

The editor is dark-mode only. It prioritizes keyboard shortcuts and minimal clicks. Every action that used to require a dialog box now uses an inline form. The interface is designed for long editing sessions without eye strain.

See Also

Related [The reader that displays your work](#)

Prerequisite [The format you are editing](#)

Example [Step-by-step tutorial](#)

In This Section

- [The Rich Text Toolbar](#)
- [Managing Children and Connections](#)
- [The Relationship Graph](#)
- [Command Palette and Shortcuts](#)

The Rich Text Toolbar

Intermediate · 5 min · toolbar, wysiwyg, formatting, keyboard

WYSIWYG editing with a formatting toolbar. What you see is what you get — Markdown is generated on save.

The editor uses a contenteditable div with a toolbar, not a raw Markdown textarea. You format text visually, and the editor converts it to Markdown when saving.

Toolbar Buttons

B — Bold (Ctrl+B)

I — Italic (Ctrl+I)

H2 — Heading (Ctrl+2)

H3 — Sub-heading (Ctrl+3)

Bullet list — Unordered list (Ctrl+U)

1. List — Ordered list

Quote — Blockquote (Ctrl+Q)

Link — Insert a hyperlink (Ctrl+L)

Code — Inline code

Paragraph — Reset to normal text (Ctrl+0)

Rule — Insert a horizontal rule

Link Insertion

When you click **Link** or press Ctrl+L, an inline form appears between the toolbar and the content area. Enter a URL and optional display text, then press Enter or click Insert. The editor saves your cursor position so the link goes exactly where you intended.

Paste Handling

Pasting from external sources (web pages, Word documents) strips all formatting and inserts plain text. This prevents messy HTML from polluting your Markdown output.

Code Blocks

Fenced code blocks with triple backticks round-trip cleanly through the editor. They render in a styled `<pre>` block and convert back to fenced Markdown on save.

See Also

Related [The Markdown it generates](#)

Related [More keyboard shortcuts](#)

Managing Children and Connections

Intermediate · 5 min · children, connections, inline-forms, undo

Add, reorder, and remove child links and cross-connections using inline forms. No dialogs, no popups.

Below the content editor, you will find three sections: **Children**, **Connections**, and **Media**. Each works the same way.

Adding a Child

Two options:

+ Link — links an existing node as a child. Opens a dropdown of available nodes.

+ New — creates a brand new node and adds it as a child in one step. Opens the new-node modal with "add as child" pre-checked.

Reordering Children

Each child in the list has up/down arrow buttons. Click them to change the order. The sidebar tree updates immediately.

Removing a Child

Click the **x** button next to a child. This removes the parent-child link but does not delete the child node.

Adding a Connection

Click **+ Add** in the Connections section. An inline form appears with:

A dropdown to select the target node

A dropdown to select the connection type (related, prerequisite, deeper, example, tangent)

An optional label field

Press Enter or click Add to create the connection.

Undo

All structural changes (add/remove/reorder children, add/remove connections) push to the undo stack. Press Ctrl+Z (when not typing in the content editor) to undo.

See Also

Prerequisite [How children work in the format](#)

Prerequisite [How connections work in the format](#)

Related [See the relationships visually](#)

The Relationship Graph

Intermediate · 3 min · graph, visualization, canvas

An interactive canvas visualization showing how all nodes relate to each other through children and connections.

In the right panel, click the **Graph** tab to see a visual map of your textbook's structure.

What It Shows

Nodes as circles, arranged by tree depth

Child links as solid arrows pointing from parent to child

Connection links as dashed lines, color-coded by type

The root node highlighted in amber

The selected node highlighted in blue with a glow effect

Orphan nodes grouped at the bottom

Interaction

Click a node to select it (loads it in the editor)

Hover over a node to highlight it and see its label more clearly

The graph **auto-resizes** when you resize the panel

Layout

Nodes are arranged in horizontal layers by depth. Level 0 (root) is at the top. Children are on the next level down. Orphans get their own row at the bottom.

The graph is useful for spotting:

Orphan nodes that are not connected to anything

Dense clusters of cross-connections

Sections with many children vs. sections with none

The overall shape and depth of your textbook

See Also

Related [Managing the relationships it visualizes](#)

Deeper [How the graph is implemented](#)

Command Palette and Shortcuts

Intro · 4 min · palette, shortcuts, keyboard, productivity

Press **Ctrl+K** to access every editor action from the keyboard. The full shortcut reference.

The command palette is the fastest way to do anything in the editor.

Command Palette (Ctrl+K)

Press **Ctrl+K** to open it. Start typing to filter. The palette includes:

All file operations (New, Open, Save)

Node creation

Preview

Tab switching (Meta, Preview, Graph)

All nodes in your textbook (type to search)

Use arrow keys to navigate, Enter to select, Escape to close.

Keyboard Shortcuts

File Operations

Ctrl+K — Command palette

Ctrl+S — Save

Ctrl+O — Open

Ctrl+N — New file

Ctrl+Shift+N — New node

Ctrl+P — Preview in new tab

Formatting (in content editor)

Ctrl+B — Bold

Ctrl+I — Italic

Ctrl+2 — Heading

Ctrl+3 — Sub-heading

Ctrl+U — Bullet list

Ctrl+Q — Blockquote

Ctrl+L — Insert link

Ctrl+0 — Normal paragraph

Editing

Ctrl+Z — Undo (structural changes, when not typing)

? — Shortcuts reference

Esc — Close modal / palette / inline form

Press **?** inside the editor to see this reference at any time.

See Also

Related [Toolbar formatting shortcuts](#)

Related [Reader keyboard shortcuts](#)

Technical Architecture

Advanced · 7 min · architecture, modules, events, technical

How both tools are built: module pattern, event bus, zero dependencies, static hosting.

Both tools are pure HTML, CSS, and JavaScript. No build step, no npm, no frameworks. Open the HTML file in a browser and everything works.

Module Pattern

Each JavaScript file is a self-contained IIFE (Immediately Invoked Function Expression) that returns a public API:

```
const MyModule = (() => {  
  function doSomething() { ... }  
  return { doSomething };  
})();
```

This means each file can be understood and modified independently. An AI editor can change one module without needing to read the others.

Event Bus

Modules communicate through a shared `State` object that provides `on(event, handler)` and `emit(event, payload)`. No module calls another directly (except `State`).

Explainer Events

`content-loaded` — a file was loaded

`navigate` — the user moved to a new node

`theme-changed` — the theme was switched

`settings-changed` — font/spacing/width adjusted

`sidebar-toggled` — sidebar opened or closed

Editor Events

`file-loaded` — a file was loaded or created

`node-selected` — a different node was selected

`node-updated` — a node's content changed

`node-added / node-deleted` — structural change

meta-updated — book metadata changed

dirty-changed — unsaved changes state toggled

Why This Matters

The event-bus architecture makes the system AI-friendly. Each module is small enough to fit in a single context window, and the event contract is the only interface between them. You can refactor the graph renderer without touching the tree panel.

See Also

Related [The shared format both tools consume](#)

Deeper [How to extend the system](#)

In This Section

- [Editor State Management](#)
- [Reader State Management](#)

Editor State Management

Advanced · 5 min · state, undo, auto-save, technical

How the editor tracks content, selection, dirty state, and undo history.

The editor's `EditorState` module is the single source of truth. It holds:

data — the full `.entropy.json` content in memory

selectedNodeId — which node is currently being edited

dirty — whether there are unsaved changes

undoStack — up to 30 JSON snapshots of previous states

fileName — the current file name for save-as

Undo System

The undo stack stores full snapshots of the `data` object (via `JSON.stringify`). Before any structural change (add, delete, duplicate, reorder, add/remove children or connections), the current state is pushed to the stack.

This is simple but effective for textbook-scale data. A textbook with hundreds of nodes might be a few hundred KB per snapshot. At 30 snapshots, that is roughly 10 MB of memory — well within browser limits.

Auto-Save

As the user types, a 400ms debounce timer saves the current field values back to the state. This means the in-memory data is always nearly current, but the file on disk is only updated when the user explicitly saves.

Word Count

The state module provides `getWordCount(nodeId)` for per-node counts and `getWordCount()` for the total across all nodes. Both count words in title, summary, and content fields.

See Also

Related [Compare with reader state](#)

Reader State Management

Advanced · 5 min · state, persistence, localStorage, technical

How the reader tracks navigation history, visited nodes, reading position, and user preferences.

The reader's `State` module manages:

data — the loaded `.entropy.json` content

currentNodeId — which node is being displayed

history — a stack of visited node IDs (for the Back button)

visited — a Set of all node IDs the user has seen

theme — current theme name

fontSize / lineHeight / contentWidth — reading preferences

Persistence

Several values are saved to `localStorage` and restored across sessions:

Theme — key: `entropy-theme`

Font settings — keys: `entropy-font-size`, `entropy-line-height`, `entropy-content-width`

Visited set — key: `entropy-visited-{book-slug}` (per-book)

Reading position — key: `entropy-position-{book-slug}` (per-book)

The book slug is derived from the title, so two books with different titles have independent progress tracking.

Linear Order

The `getLinearOrder()` function returns all node IDs in depth-first tree order, with orphans appended at the end. This powers the Prev/Next navigation buttons and ensures every node is reachable through linear traversal.

See Also

Related [Compare with editor state](#)

Related [The user-facing progress features](#)

Quick Start: Your First Textbook

Intro · 8 min · tutorial, getting-started, walkthrough

A step-by-step walkthrough: create a textbook, add nodes, build structure, save, and view in the reader.

Follow these steps to create your first Entropy textbook:

Step 1: Open the Editor

Open `editor/index.html` in your browser. You will see the empty state.

Step 2: Create a New File

Click **New** in the header (or press Ctrl+N). A blank textbook is created with one node called "Introduction."

Step 3: Edit the Root Node

The Introduction node is selected automatically. Fill in:

Title — your textbook's opening section name

Summary — a brief description (appears on parent cards)

Content — write your opening text using the rich text toolbar

Step 4: Add More Nodes

Click **+ Node** in the header (or Ctrl+Shift+N). Give your new node a title — the ID is generated automatically. Check "Add as child of current node" to link it as a sub-section.

Repeat for each section of your textbook.

Step 5: Add Connections

Scroll to the **Connections** section in the editor. Click **+ Add**, choose a target node and connection type, then click Add.

Step 6: Set Metadata

Switch to the **Book Meta** tab in the right panel. Set your title, author, and description.

Step 7: Save

Press Ctrl+S. Your browser downloads a `.entropy.json` file.

Step 8: Preview

Press Ctrl+P to open a self-contained preview in a new tab. Or open `explainer/index.html` and drag your saved file onto it to see the full reader experience.

That is it. You have a working interactive textbook.

See Also

Deeper [Full editor documentation](#)

Related [Understand the format you just created](#)

Related [Full reader documentation](#)

Extending the System

Expert · 10 min · extending, customization, themes, development

How to add new themes, new node fields, and new connection types. The system is designed to grow.

Entropy is designed to be extended without rewriting core logic.

Adding a New Theme

Open `explainer/css/themes.css`

Add a new `.theme-yourname` block with all the CSS custom properties

Open `explainer/js/themes.js` and add `'yourname': 'Display Name'` to the `themeMap`

That is it. The theme picker, persistence, and all components will use your new theme automatically.

Adding a New Node Field

Add the field to `shared/schema.json`

In the editor: add an input field in `editor/index.html`, bind it in `editor.js` (in `loadNode` and `setupAutoSave`)

In the reader: render the new field in `explainer/js/renderer.js`

Because modules communicate through events, you only need to touch three files.

Adding a New Connection Type

Add the type to the enum in `shared/schema.json`

Add a CSS class `.conn-type-yourtype` in `explainer/css/components.css`

Add it to the connection type dropdown in `editor/index.html`

The graph, connection pills, and inline forms all pick it up automatically because they read the `type` field dynamically.

Adding Media Types

The `explainer/js/media.js` module has a switch statement for media types. Add a new case to support your format. The editor's media form already has a free-text type dropdown you can extend.

See Also

Prerequisite [Understand the architecture first](#)

Prerequisite [The schema you are extending](#)